Introduction to Hardware Design
Basic Digital Logic: Figures and Code Snippets

Profs. Sundeep Rangan, Siddharth Garg

0.1 ReLU with a linear input

We consider implementing the ReLU function:

$$y = \max\{ax + b, 0\}$$

where x, a, and b are integer inputs. This function arises commonly in machine learning. We first consider a purely combinational implementation of this function. A possible SystemVerilog implementation is as follows:

```
module relu_lin (
  input logic signed [31:0] x,
  input logic signed [31:0] a,
  input logic signed [31:0] b,
  output logic signed [31:0] y
);
  always_comb begin
    logic signed [31:0] mult_out, add_out;
    mult_out = x * a;
    add_out = mult_out + b;
    y = (add_out > 0) ? add_out : 0;
  end
endmodule
```

When this module is synthesized, a potential block diagram is as shown in Figure 1.

0.2 Two input circuit

Next, we consider a simple two input circuit with the following SystemVerilog code:

```
xsq = x * x;
r = (x > 0) ? x : 0;
y = xsq + r;
```

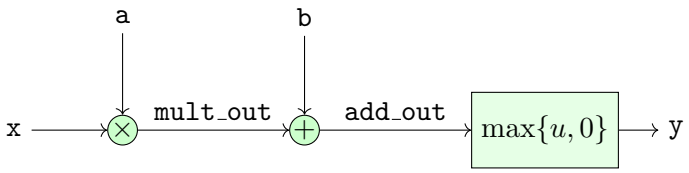


Figure 1: Fully combinational block diagram for ReLU with a linear input.

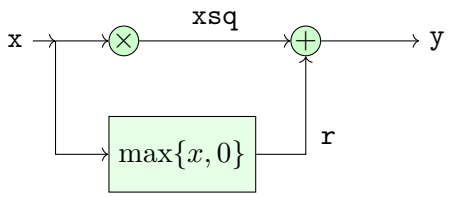
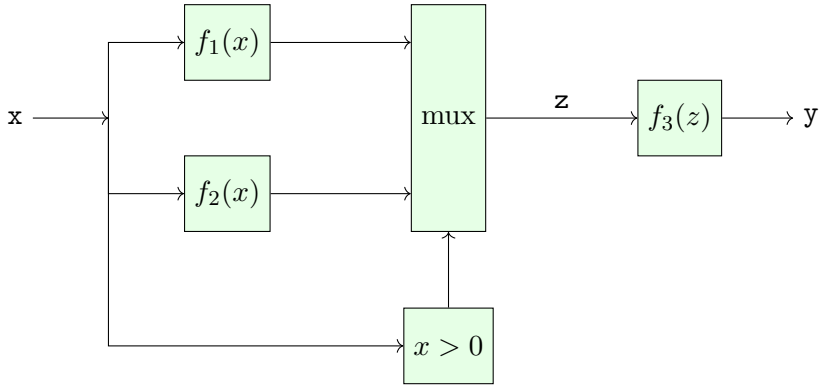


Figure 2: Multi-path circuit.



0.3 Block diagram for a multiplexer

```
// mux delay = 1 ns
if (x > 0) begin  // delay = 0.5
  z = func1(x);  // delay = 4
end else begin
  z = func2(x);  // delay = 2
end
y = func3(z);    // delay = 1.5
```

0.4 Register

Consider the following SystemVerilog code for a register:

```
always_comb begin
```

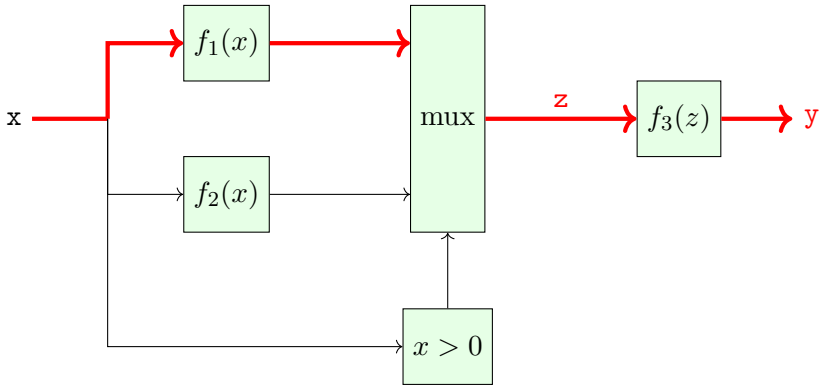


Figure 3: Block diagram for a multiplexer highlight the critical path

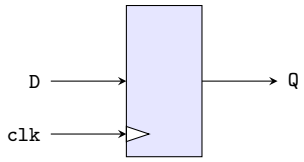


Figure 4: Basic register

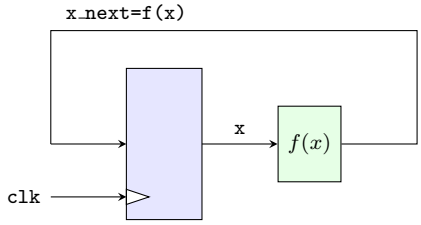


Figure 5: Basic register block diagram of $x \leftarrow f(x)$

```
    x_next = f(x);
  end
  always_ff @(posedge clk) begin
    x <= x_next;
  end
```

0.5 Two-Variable System

Consider two-variable system:

```
  always_comb begin
    z0 = func1(x0);
    z1 = func2(x1);
    x_next0 = func3(z0, z1);
    x_next1 = func4(z0, x1);
  end
  always_ff @(posedge clk) begin
    x0 <= x_next0;
    x1 <= x_next1;
  end
```

Register-to-register paths are:

```
x0 -> func1 -> func3 -> x0
x0 -> func3 -> x1
x1 -> func2 -> func3 -> x0
x1 -> func4 -> x1
```

0.6 Minimum Hold Time

Consider the following SystemVerilog code:

```
always_ff @(posedge clk) begin
  x0 <= x;
  x1 <= x0;
end
```

This is a simple delay line:

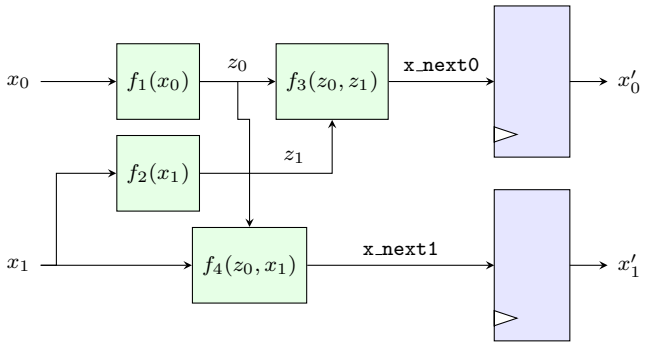


Figure 6: Simple two variable system

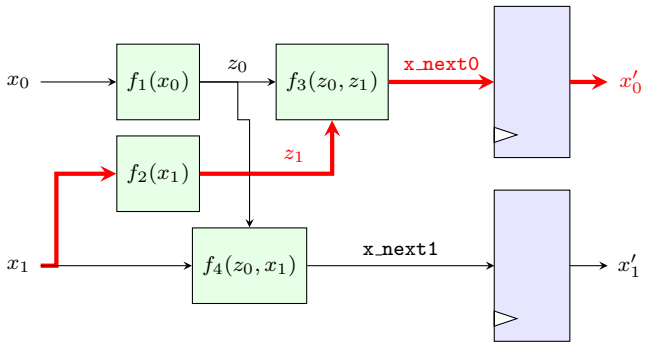


Figure 7: Critical path in two variable system

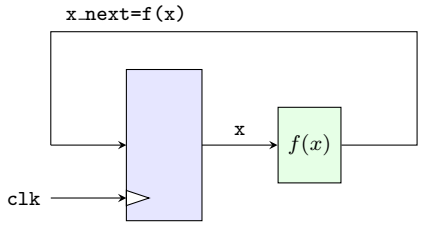


Figure 8: Basic register block diagram of $x \leftarrow f(x)$

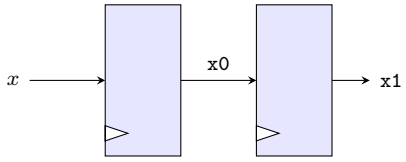


Figure 9: Simple delay line